Scalable and Resilient Storage Tier for Azure SQL Hyperscale

Alejandro Hernandez Saenz
Krystyna Reisteter
Sarika Iyer
ahsaenz@microsoft.com
Krystyna.Reisteter@microsoft.com
saiyer@microsoft.com
Microsoft Corporation
Redmond, USA

Yu Wang
Shweta Raje
Swati Roy
yuwng@microsoft.com
shraje@microsoft.com
swroy@microsoft.com
Microsoft Corporation
Redmond, USA

Bhupesh Chawda
Kashish Goyal
Vishnu Das
bhchawda@microsoft.com
goyalkashish@microsoft.com
vishnudas@microsoft.com
Microsoft Corporation
Bengaluru, India

Kinshuk Chopra
Rishita Chauhan
kichopra@microsoft.com
richauhan@microsoft.com
Microsoft Corporation
Bengaluru, India

Prashanth Purnananda
Hanuma Kodavalla
praspu@microsoft.com
hanumak@microsoft.com
Microsoft Corporation
Redmond, USA

Abstract

Azure SQL Database's Hyperscale architecture disaggregates compute, log, and storage to enable independent scaling for databases up to 128TB. While this design offers elasticity, it introduces latency and resiliency challenges due to remote IO between Page Servers and Azure Storage. To address these issues, this paper proposes a redesigned storage tier that enhances performance, scalability, and resiliency for OLTP workloads.

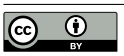
The first strategy, file anatomization, stripes a logical file into smaller physical files, mitigating per-file IO limits and decoupling logical and physical layouts for improved throughput. The second strategy, asynchronous write-behind, which aggregates write operations before committing to Azure Storage. This approach not only safeguards against service outages and enables efficient defragmentation but also reduces the number of write IOs, thereby improving read performance for page retrieval operations. Together, these innovations establish a robust foundation for distributed database storage, delivering significant gains in elasticity, fault tolerance, and overall system efficiency. This paper also presents experimental findings and the impact of deploying these improvements across Azure SQL Hyperscale production databases.

CCS Concepts

• **Information systems → Data management systems**; **Storage architectures**.

Keywords

Scalability, Cloud DB, Hyperscale, Storage V2, Azure Storage, Compute, Page Server, IOPS, Write-Behind RBPEX



This work is licensed under a Creative Commons Attribution 4.0 International License.
SIGMOD Companion '26, Bengaluru, India
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2450-3/2026/05
https://doi.org/10.1145/3788853.3803083

ACM Reference Format:
Alejandro Hernandez Saenz, Krystyna Reisteter, Sarika Iyer, Yu Wang, Shweta Raje, Swati Roy, Bhupesh Chawda, Kashish Goyal, Vishnu Das, Kinshuk Chopra, Rishita Chauhan, Prashanth Purnananda, and Hanuma Kodavalla. 2026. Scalable and Resilient Storage Tier for Azure SQL Hyperscale. In *Companion of the International Conference on Management of Data (SIGMOD Companion '26), May 31-June 05, 2026, Bengaluru, India.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3788853.3803083

1 Introduction

The Hyperscale edition of Azure SQL Database [2][7] employs a cloud-native, disaggregated architecture that enables independent scaling of compute and storage resources, thereby supporting a broad spectrum of traditional and modern application workloads. As depicted in Figure 1, the architecture comprises several core components, including Compute, Page Servers (PS) for storage, a log service for transactional consistency, and Azure Storage for durable persistence[3]. The Transaction tier, comprising of Compute, serves as the client-facing interface, executing queries and managing user interactions. The Log tier, containing Log Service, oversees all transaction log records related to data changes from the primary Compute. Page Servers own disjoint sets of database pages, keep them current by applying log records, and serve pages to Compute on demand; Azure Storage [7] persists the page files (blobs) that Page Server manages. This architecture scales to multi-terabyte databases while relying on remote IO along two boundaries: Compute <-> Page Server and Page Server <-> Azure Storage. For this work, focus is on the latter — Page Server <-> Azure Storage — because it dominates durability and bulk data movement paths for OLTP workloads [9][8]. Hyperscale's storage layer also leverages blob snapshots for backup and point-in-time restore (PITR): The service takes periodic snapshots of Page Server data files and selects the closest snapshot to fulfill PITR, promoting copies when needed. Conceptually, snapshots are lightweight, copy-on-write views of large page blobs; operationally, frequent snapshotting of large per-Page Server files — historically on the order of 128 GB to 1 TB per Page Server data file — can concentrate IO on individual

blobs and stress remote storage limits under heavy churn. This context motivates our work: improving the efficiency and resilience of the Page Server <-> Azure Storage path without changing transaction semantics or the Compute <-> Page Server access model.

In the prior storage tier layout, each logical file maps to a single large blob, which creates per-blob throughput ceilings and increases the likelihood of throttling when remote storage limits are exceeded. Under sustained OLTP workload and snapshot activity, this manifests as higher Page Server <-> Azure Storage latency, slower log redo and longer seeding/restore times. Additionally, SQL Server's 8 KB page size leads to fragmented writes at the blob layer, amplifying metadata overhead and reducing write coalescence. Finally, coupling Page Server writes synchronously to both the SSD cache Resilient Buffer Pool Extension (RBPEX) [10] and Azure Storage elevates the write critical path and reduces tolerance to transient storage faults. These limitations are characteristics of the prior storage layout; the present paper proposes a redesigned tier to address them.

Two complementary strategies are introduced that optimize the Page Server <-> Azure Storage path while preserving Hyperscale's disaggregated architecture. The first strategy is file anatomization which restructures large logical files into multiple smaller physical files (striped blobs). By decoupling logical layout from physical placement, this spreads IO across blobs and mitigates per-blob limits. The second strategy, asynchronous write-behind in RBPEX, aggregates write operations and commits them asynchronously to Azure Storage. Async Write-behind reduces the number of write IOs and lowers blob write contention thereby improving read service for page retrieval (GetPage from Compute tier) during concurrent load. These strategies target performance, scalability, and resiliency for OLTP workloads at Hyperscale.

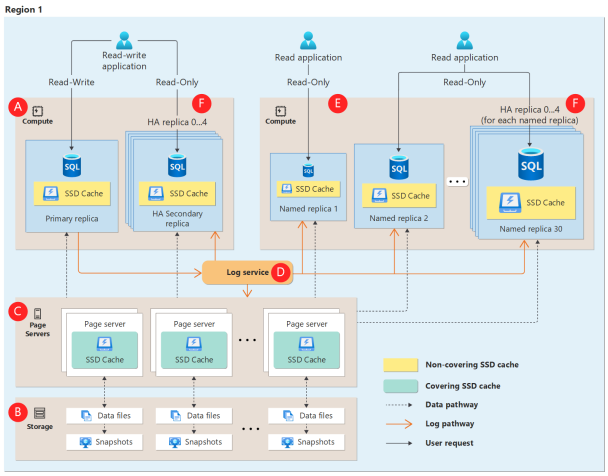


Figure 1: Hyperscale Architecture

2 Background

2.1 Architectural Overview of Hyperscale

The Hyperscale service tier in Azure SQL Database represents a cloud-native evolution of traditional database architecture, designed to overcome the limitations of monolithic systems. It decouples compute, log, and storage components into independently scalable units, enabling seamless expansion of resources to meet dynamic workload demands. This disaggregated model allows Hyperscale databases to scale up to 128 TB, far exceeding the capacity of other Azure SQLDB offerings. As depicted in Figure 1, the architecture comprises of four primary components: Compute Primary and Secondaries, Page Servers, the Log Service, and Azure Storage. Each component is optimized for specific roles—Compute handles query execution and user interaction, Page Servers manage data page availability, correction and delivery, the Log Service ensures transactional consistency, and Azure Storage provides durable persistence[2].

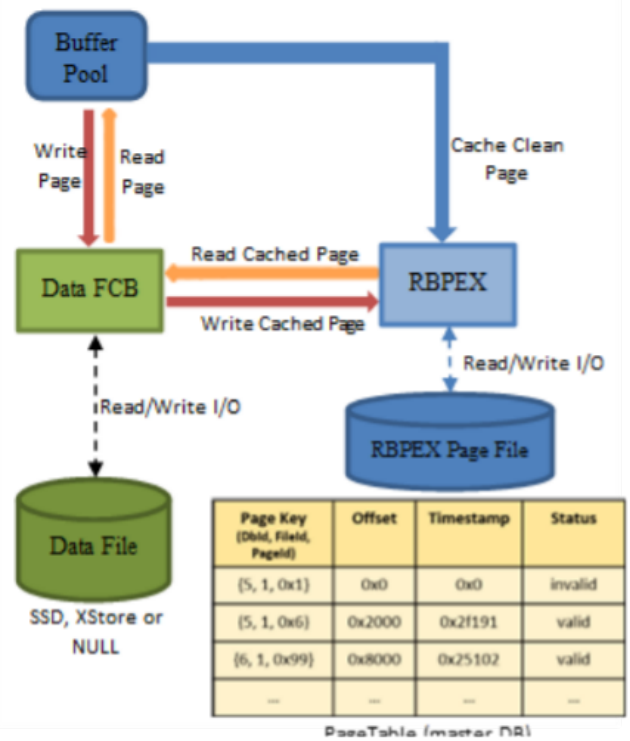


Figure 2: Page Read-write path on the Page Server and segmentation in RBPEX

2.2 Storage Tier Composition and Functionality

At the heart of Hyperscale's scalability lies its innovative storage tier. Page Servers are the primary storage component that serves pages to the Compute, with these pages securely persisted in Azure Storage to guarantee durability. They retrieve data pages from storage blobs and cache them in local SSD (RBPEX) for efficient access. Page Servers apply transaction logs received from the Log Service to maintain page consistency. Azure Storage, built on blob infrastructure, serves as the long-term repository for data pages, offering redundancy and durability. Azure Storage is a cheap remote storage service but there is no guarantee of throughput and latency at an acceptable price point. To offset the performance constraints

of Azure Storage, RBPEX was introduced as a recoverable SSD cache which exists on both Compute and Page Servers.

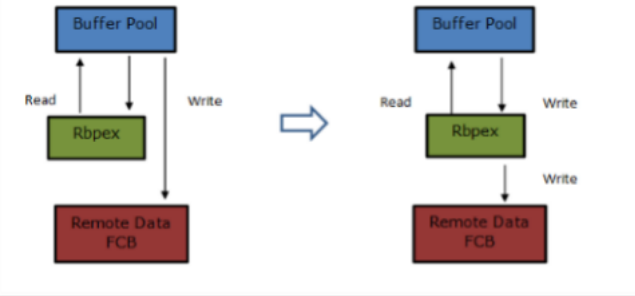


Figure 3: Old and new paths to flush pages

2.3 RBPEX Background

The Resilient Buffer Pool Extension (RBPEX) functions as a caching mechanism for pages at both the compute and the storage tiers. By maintaining a recoverable cache, RBPEX significantly reduces mean time to recovery (MTTR). For transient failures such as a machine reboot, recovery is significantly faster because data can be read directly from local SSD storage rather than being rehydrated from remote Azure Storage. This eliminates the latency associated with fetching cached pages from a remote tier, enabling quicker restoration of service. A shorter MTTR directly improves system availability. RBPEX is integrated with the in-memory storage engine Hekaton [4], which ensures that read IO to RBPEX is as fast as direct IO to the local SSD. Write IO to RBPEX, however, must be carefully managed: RBPEX IO cannot stall data IO, and failures in RBPEX must not compromise page integrity.

Here the primary focus is on the RBPEX deployed on Page Servers, which is backed by a local SSD file. The file has a fixed size that is a multiple of the page size (e.g., 8 KB). It stores only pages without additional metadata. Metadata is instead maintained in a Hekaton table [4] of the Page Server instance, which acts as the authoritative source for metadata management.

To track the placement of data within local storage, a metadata table is maintained that maps logical database segments to their corresponding positions in the file. This metadata is implemented as a transactional, in-memory Hekaton table [4]. When a page in the buffer pool is modified, it is marked as dirty and must be written to both RBPEX and Azure Storage (XStore) to ensure persistence. Consequently, write IO operations for dirty pages occur on both RBPEX and Azure Storage.

Page read requests from Compute nodes to Page Servers are evaluated in a tiered manner: the request first probes the Compute RBPEX; if the page is not present, a remote call is issued to the Page Servers. The Page Server then attempts to serve the request from its in-memory buffer pool; failing that, it probes its SSD-backed RBPEX; and if still not found, the request falls back to a remote read from Azure Storage (XStore).

In the prior Hyperscale implementation, SQL checkpointing flushes dirty pages to both RBPEX and the remote file hosted on Azure Storage, thereby making write IO dependent on Azure Storage latency and availability. To address this limitation, RBPEX introduces a Write-Behind mechanism. Instead of forwarding every write IO to the Azure Storage blob, RBPEX captures dirty pages in an in-memory bitmap and collates their propagation. A background process then asynchronously migrates these pages from RBPEX to the Azure Storage blob. This approach decouples local checkpointing, which flushes dirty pages from the buffer pool to RBPEX, from remote checkpointing, which asynchronously propagates RBPEX updates to Azure Storage (Figure 3).

2.4 Limitations and Challenges of the Storage Tier

The Hyperscale architecture in Azure SQL Database makes several design choices that overlook the inherent limitations of the underlying remote storage layer—Azure Storage. Notably, the fixed file size of 1 TB represents a substantial blob size for remote storage systems. When subjected to intensive read/write operations, such large blobs concentrate IO traffic, which can quickly exhaust Azure Storage's IO limits. Once these limits are breached, the system experiences IO throttling, leading to degraded performance from Azure Storage. Furthermore, frequent snapshot creation introduces significant overhead on the storage subsystem because each snapshot operation requires capturing a consistent point-in-time image of the data. For large blobs, this process involves duplicating metadata and maintaining version chains, which increases random IO activity. As snapshots accumulate, the system must sustain additional writes for both the active data and the snapshot delta pages, amplifying IOPS consumption. This compounded workload not only elevates latency but also destabilizes throughput under heavy transactional pressure, ultimately reducing reliability for large blobs.

Below are the undesirable consequences of the above design limitations:

2.4.1 Failover Handling. Failover incurs additional latency for large data file sizes. In the prior storage tier layout, each 1TB file is stored in Azure Storage and managed by a single Page Server instance. This means that if the SQL instance hosting that 1TB Page Server file fails, a new SQL instance must be provisioned to replace it. This triggers the RBPEX seeding process, wherein the local SSD (RBPEX) is reconstructed by reading the entire data file (1 TB) from Azure Storage. This brings two issues – first, seeding such a large file can easily exceed the blob limit; second the seeding time can be as long as hours for a single Page Server replica. During this long seeding phase, the Page Server continues to serve read requests and apply modifications from the transaction log stream. Empirical observations indicate that this process leads to a substantial drop in query throughput for read-heavy workloads and impairs the Page Server's ability to keep pace with incoming log changes.

To mitigate this limitation, a logical solution involves reducing the size of the files managed by each Page Server. However, logical file sizing decisions are customer-controlled, and many existing databases — particularly those migrating from non-Hyperscale tiers — already contain several files of size 1TB. Requesting customers to manually shrink these files prior to migration is impractical, often requiring several days of continuous shrink operations. Given this constraint, there should be control over the portion of the file that

each Page Server is responsible for. So internally one big file (as seen by the customer) is split into different, independently maintainable file-slices of the database.

The key part is that each file-slice can be hosted by a different Page Server, so the amount of data that "it owns" is variable and determined by the Compute. This gives us the ability to have significantly faster Page Server re-creation, since the data being managed by each Page Server is small. This also allows the Page Server to sustain faster log-redo, since one Page Server is only responsible for maintaining a portion of the file, any modifications to pages not owned by that Page Server are ignored.

2.4.2 Resiliency Consideration. Another concern arises from outages and performance limitations with Azure storage accounts hosting Page Server remote files, hindering write speed. The prior architecture, where data write is a single long IO writing to both RBPEX and Azure Storage, creates a dependency on Azure Storage availability and performance. The resolution involves decoupling the IO write path for RBPEX and Azure Storage, ensuring the Page Server's normal functioning remains unaffected even during Azure Storage downtimes. This is achieved through a Write-Behind approach, where RBPEX no longer forwards all write IO operations to the underlying Azure Storage blob. Instead, RBPEX captures dirty bits in an in-memory buffer, tracking locally modified pages ahead of the Azure Storage file. A secondary process asynchronously moves these pages from RBPEX to the Azure Storage blob in the background. By segregating the IO going to RBPEX and introducing a background task for Azure Storage data transfer, even in the absence of Azure Storage, write operations to RBPEX (SSDs) remain available, providing a high level of resiliency for write operations.

2.4.3 IO Limits. The limitation with OLTP/IoT workloads that modify many little rows is that the number of changes for a given Page Server (write modifications) overwhelms the IO throughput of the remote Azure Storage file. Additionally, there is no control on the size of the files of existing databases, as they already exist in SQL DB or on On-Prem systems, which means lots of IOs can fall into the same remote file. Azure Storage has approximately 500-1000 IOs bandwidth per blob for Standard storage[3]. Once the limit is crossed, it starts throttling incoming requests.

To increase the IO throughput, one could put the remote file on a higher storage tier (Premium) [6], but that has significant impact on Cost of Goods Sold (COGS), plus there are other limitations like Snapshots that are not supported. Instead, it was decided to split the single Azure Storage file into multiple smaller files called Stripes. Each Stripe corresponds to an Azure Storage file, with its own 500 IO's cap, increasing the overall throughput. To alleviate the IO pressure on a single remote file, another approach that follows is to combine multiple writes to Azure Storage allowing bigger IOs and less overall write operations, which reduces the IOPS per Azure Storage file and defragments it.

Both these storage optimization strategies improve remote storage IO performance.

From the above, the following solutions naturally flow:

- Segregation of logical file concepts from physical file.
- The IOs to RBPEX vs IOs to Azure Storage need to be independent.
- Merge multiple IOs from RBPEX to Azure Storage into one.

Details of the solutions are discussed below.

Table 1: Cell configuration values

Cell Configuration	Value
Stripe Count	4
Stripe Size	4 GB
IOStrideInKBytes	1024 KB

3 Logical-Physical File Separation for IO Efficiency

This section outlines a detailed technical overview of how the logical and physical storage layers are decoupled in Hyperscale storage tier to enhance the remote IO performance. The separation is achieved through the following key technical modifications:

- Partitioning a single logical file into file slices, each hosted by an individual Page Server.
- Partitioning each file slice further into cells, and subsequently into stripes.
- To create a single I/O operation, data is written in strides in a round-robin fashion across multiple stripes.
- Distributing a single logical file over multiple Page Servers through endpoint mapping.

For clarity, a detailed description of each concept is provided below.

3.1 Cell

A cell is defined as the indivisible unit of storage in the proposed architecture. While in the prior Hyperscale system this role is served by an Azure Storage blob, the redesigned model introduces the cell as the fundamental storage unit.

Each cell is composed of a fixed, predefined number of stripes, with each stripe having a uniform size and being stored in a separate Azure Storage blob. Consequently, the size of a cell is fixed, determined by the stripe size and stripe count.

Cells are organized to maximize the write throughput while maintaining read efficiency. IO operations are performed in fixed size strides, with successive strides alternating across stripes. This prevents sequential write patterns from overloading any single stripe, while the stride length is chosen to ensure sufficiently long read runs, reducing the need for frequent IO splitting or merging. Table 1 summarizes the typical cell configuration values.

The cell described above has a total size of 16GB. With sequential writes switching to the next stride at 1MB boundaries. IO operation that crosses stride boundaries requires splitting and merging of reads and writes.

Cell concepts directly address limitations observed with Azure Storage in the current Hyperscale architecture. At the base of the storage stack, a configurable storage primitive is required to overcome the IOPS constraints of Azure Storage blobs, even for small data files. Cells and stripes serve as the foundational building blocks for striped files and are described below.

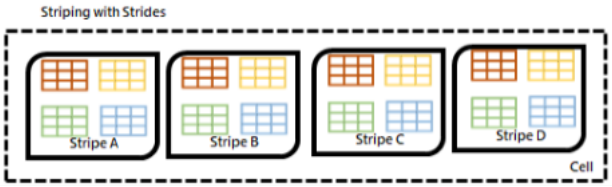


Figure 4: Striping with strides

3.2 Stripe

A striped file is part of a cell, and the cell has a fixed number of striped files. Although a cell is made of stripes, a stripe is not considered an indivisible unit of storage. As will be shown later, the content of a stripe cannot be interpreted in isolation without the other stripes that make up a cell. This is because in Storage V2, data within a cell is interleaved across its stripes in a round-robin pattern at stride granularity, so each stripe holds only every Nth fragment of the logical address space, not a contiguous block. This means a single stripe's content is meaningless without the other stripes and the cell metadata needed to reassemble the original logical page order, making the stripes interdependent by design for I/O parallelism.

3.3 Stride

Each Cell is written into striped files in round-robin strides (Figure 4). Striding the striped files provides better performance for read-ahead data, since every stride will use a different striped file from the cell.

3.4 Slice

A slice refers to a contiguous range within a SQL logical file, with ownership assigned to a single Page Server. When the Compute requests a page, it consults the connection map to determine which Page Server hosts the slice containing the page and directs the requests accordingly.

A Slice acts as a container for Cells by adding metadata which makes IO operations possible. Slice usually contains only one cell, but they can contain multiple cells in a way that is transparent to any layer above the Slice.

As defined above, cells cannot be used without knowing their configuration: without knowing the stride for instance, there is no way to know when a read operation should be split (because it spans the stride boundary). Also, without knowing which stripes exactly make up a cell, there is no way to read the cell content at all. Slices store the metadata that the cells are missing – so in a very real sense, Slice is just Cells plus metadata. There are several other functions that get associated with a Slice, but this simplification (Slice = cells + metadata) is a useful shortcut.

Stripes enable parallelism and improved IO throughput by distributing data across multiple Azure Storage blobs within a cell, mitigating performance bottlenecks caused by large monolithic files. However, stripes alone lack the metadata required for logical interpretation and coordinated access. Slices complement this by encapsulating cells with metadata, mapping logical file ranges to Page Servers and enabling precise IO operations. Together, stripes

```
CELLS:
{
   {
      Schema Version:1,
      StripeCount : 4,
      IOStrideInKBytes : 1024,
      StripeSize: 4GB,
      StripeFiles: {F1, F2, F3, F4},
      FirstPageId: <PageID>,
      ForeignApplyLSN: <LSN>,
      GlobalFileHeader: 0x0b39fd04…
   }
}
```

Figure 5: Cell layout

provide physical distribution while slices ensure logical organization and operational consistency. The use of such fine-grained building blocks enhances scalability and flexibility. For instance, a large file can be partitioned into smaller segments, each managed by a distinct Page Server instance. Conversely, multiple small or inactive files can be consolidated under a single Page Server instance, enabling dynamic and efficient resource utilization in Storage V2.

The metadata block is a JSON document containing an array of cells. This example shows just one cell (Figure 5). The cell properties are captured here, in addition to the actual location of the stripes that make the cell (F1..F4).

But there are also properties that do not pertain to the cell itself, but rather to the foreign file that rests on top of this striped file. For instance, this is where the properties related to database checkpoint would get stored. The header page (Global File Header) for the logical file represented by this striped file is stored here as well.

The Global File Header read/write paths will be modified to maintain the metadata in the Slice metadata block instead of storing it in the header page of the underlying file. This is required to slice a file of arbitrary size into a collection of equal-sized cells. During this separation, the resulting slices no longer retain the ability to each own a header page. Since they continue to be kept up to date via log redo and need to continue to checkpoint and behave like regular SQL files, the Global File Header needs to be maintained for Hyperscale specific fields.

The Slice layout will look like Figure 6. Figure 7 shows Stripe geometry inside each cell for a single logical file.

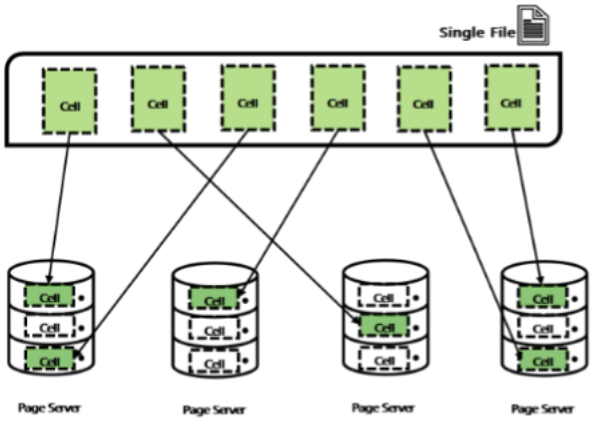


Figure 6: Slice layout

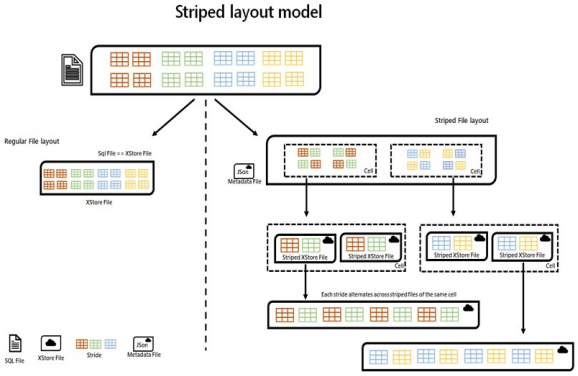


Figure 7: Striped layout mode

3.5 Why This Approach Solves the Identified Limitations

The redesigned Hyperscale storage architecture fundamentally addresses the latency, resiliency, and IO throughput issues highlighted earlier through logical-physical decoupling and granular data distribution.

3.5.1 Reduced Replica-Recreation Latency. By partitioning large logical files into smaller, independently hosted slices and cells across multiple Page Servers, the failure of one instance no longer requires re-provisioning or reloading a full 1TB file. Recovery is now limited to affected slices, enabling faster failover and significantly reduced replica re-creation times.

3.5.2 Improved Resiliency and Write Path Efficiency. The decoupling of logical data control from physical storage reduces the dependency on any single storage blob for write operations. Writes are distributed across multiple stripes and cells, which mitigates the impact of transient slowdowns in individual blobs by allowing other stripes to continue processing. However, complete unavailability of a blob can still affect write availability. The new storage tier optimizations primarily address performance variability, not full fault isolation.

3.5.3 Higher IO Throughput for OLTP/IoT Workloads. The segmented cell and stripe model introduces parallelism in IO by spreading small block writes across multiple blobs instead of funneling through a single large file. This design eliminates the single-point IO saturation, enabling aggregate throughput to scale nearly linearly with the number of participating stripes up to the point where other system bottlenecks (e.g., network bandwidth or CPU) begin to dominate.

In summary, the new architecture transforms the monolithic file design into a distributed, metadata-aware storage topology. This results in faster recovery, consistent performance underload, and more predictable scalability directly resolving the limitations identified in the existing Hyperscale storage design.

4 Write-Behind RBPEX

Write-behind RBPEX is an enhancement to the covering RBPEX. A covering RBPEX refers to a mode where the RBPEX acts as a persistent SSD-based cache layer that fully covers all data pages rather than caching subset of pages. The central idea is simple: when Write-Behind for RBPEX is enabled, RBPEX no longer forwards all write IO operations to the underlying Azure Storage blob. Instead, RBPEX captures in an in-memory buffer all the dirty bits tracking which pages have been modified locally ahead of the underlying Azure Storage file. Using this information, a secondary process moves pages asynchronously from RBPEX back into the Azure Storage blob in the background.

The principal features of the Write-Behind RBPEX architecture are outlined as follows.

- Write IO operations to Azure Storage files are bifurcated and delegated to a distinct background task to enhance system efficiency.
- Individual IO requests are aggregated prior to being committed to Azure Storage, optimizing write operations and resource utilization.
- Key information required for rapid Page Server recovery is securely stored on the local RBPEX instance, ensuring high resilience.

To understand Write-Behind RBPEX, it is important to understand the RBPEX file. Section 2.3 provides a high-level overview of what RBPEX file is.

4.1 Storage Tier Composition and Functionality

The RBPEX Write-Behind implementation consists of two essential components:

- Write-behind Checkpoint – A dedicated thread that flushes pages from RBPEX to the foreign file.
- Write-behind Recovery – Invoked when the SQL server hosting the Page Server fails, leading to a loss of in-memory state.

4.1.1 Write-Behind Checkpoint. Write-Behind checkpoint asynchronously propagates pages from RBPEX to remote Azure Storage. To enable this, RBPEX maintains an in-memory buffer (a dirty-page bitmap), where each bit corresponds to a page. The bit is set for every page that has been dirtied — i.e., written to RBPEX but not yet persisted to Azure Storage. This mechanism distinguishes

the Page Server's local checkpoint (pages written from the buffer pool to RBPEX) from the foreign checkpoint (the Write-Behind checkpoint).

To understand the operation of a Write-Behind checkpoint, it is necessary to examine how metadata is maintained across RBPEX and Azure Storage. At the per-database level, two key categories of metadata are tracked:

- Local log-apply metadata, which records the progress of the local checkpoint writing to RBPEX.
- Remote log-apply metadata, which holds details about foreign file checkpoints writing to remote Azure Storage file.

The log-apply metadata primarily pertains to checkpoint-related information stored in the Global File Header (GFH), including the foreign redo LSN, foreign redo timestamp, and related values. Two versions of log-apply information are maintained: (i) the remote values stored in the GFH as well as in Hekaton, and (ii) the local log-apply values retained in a Hekaton table within RBPEX, also referred to as the RBPEX-level Apply Information (RAI) (Table 2).

Table 2: List of RAI values stored on RBPEX

Metadata	Description
AppliedLsn	The checkpoint LSN for the foreign log-apply. All pages persisted have a PageLSN ≥ AppliedLsn.
AppliedTime	The time associated with AppliedLsn used for snapshots.
OldestLsn	The oldest LSN for the foreign database. Log older than OldestLsn does not need to be preserved.
FileSize	Database file size in PAGESIZE units.

As part of the Write-Behind mechanism, dirty pages are written from RBPEX to Azure Storage by a dedicated background thread. The Applied LSN information, stored in the local RBPEX-level Apply Information (RAI), serves the role of the Page Server's local checkpoint.

RAI maintains two states: LocalState and RemoteState. LocalState tracks the progress of log application and checkpointing at the RBPEX level, whereas RemoteState denotes the state persisted to remote storage (Azure Storage) through the Global File Header (GFH). The process begins by taking a snapshot of the current LocalState. Subsequently, LocalState is updated and advanced as part of the Page Server's local checkpoint. This updated LocalState is first used to flush page updates to Azure Storage, after which the Global File Header (GFH) is modified accordingly. Only after this step succeeds, the RemoteState is updated to reflect the changes.

This ordering guarantees the invariant that the LocalState is always greater than or equal to RemoteState. Maintaining this assertion is critical to ensure that recovery can always proceed successfully and that no page corruption occurs. Figure 8 illustrates this process.

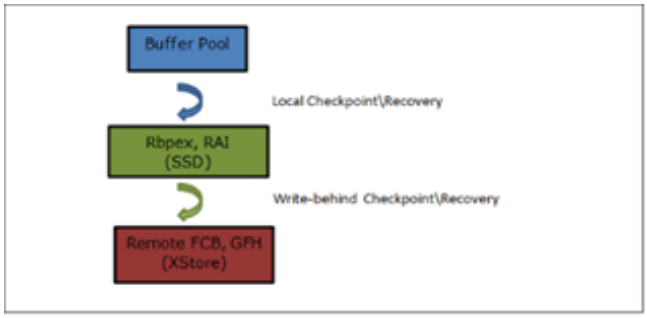


Figure 8: Local and Remote checkpoints

A new periodic process synchronizes dirty pages between RBPEX and Azure Storage to maintain consistency across the two. At the same time, the page modified bitmap is refreshed by clearing entries for pages updated up to the local applied LSN, leaving only those modified after this point marked as dirty.

The thread would start by reading the RAI (RBPEX Apply Information) and then scanning the page modified bitmap and extracting one subset of bits at a time for movement between RBPEX and Azure Storage.

The Write-Behind checkpoint thread processes modified pages by selecting specific bits from the bitmap and transmitting the corresponding pages to Azure Storage in dynamically sized chunks. In contrast to traditional flush mechanisms, this approach eliminates the need for page latching, instead utilizing Write-behind bitmaps for concurrency control. If an IO error occurs, the bits corresponding to the affected pages are set back into the modified page bitmap, ensuring these pages remain flagged for subsequent re-write.

As previously noted, latch usage is avoided during write operations to minimize interference with concurrent foreign redo operations on the same pages. The system employs three distinct bitmaps to prevent conflicting actions while pages are written to remote storage. Additionally, a PagesModified bitmap is maintained, which the Write-Behind thread references when flushing pages to remote storage.

The strategy for avoiding read-write conflicts using bitmaps is as follows:

- PagesInReadIO bitmap - The Write-Behind thread tracks pages under read operations within the PagesInReadIO bitmap. Bits are set before initiating reads and cleared once IO completes.
- PagesInDoubt bitmap - For pages being written to RBPEX, the writer maintains their status in the PagesInDoubt bitmap while writes are in progress.
- PagesModified bitmap - Upon completion, the relevant bits are cleared from PagesInDoubt and set to the PagesModified bitmap.
- PagesReadFailedIO bitmap - If the writer detects that a completed page IO still has its Read IO bit set, it sets the corresponding bit in the PagesReadFailedIO bitmap.
- Following a read operation, the Write-Behind thread verifies whether the just-read page's bit is set in the PagesReadFailedIO bitmap; if so, it triggers a read retry. It also checks that

the page does not have its bit set in PagesInDoubt, issuing a retry if necessary.

This methodology enables the issuance of RBPEX page reads via the Write-Behind thread without obstructing the foreign redo process, thereby safeguarding foreign redo performance.

Once the write-behind checkpoint thread completes and a full scan has flushed all remaining dirty pages, the checkpoint is finalized by updating and persisting GFH and associated checkpoint metadata.

4.1.2 Write-Behind Recovery. The Write-Behind recovery (Figure 9) utilizes the RBPEX-level Apply Information (RAI) stored on RBPEX. This process is supported by checkpoint LSNs maintained both locally (LocalState) and remotely (RemoteState), which track the progress of checkpointing in local and remote environments, respectively. These checkpoints are instrumental for ensuring successful checkpoint closure for flushing database pages and enabling efficient recovery when required. Recovery of the Page Server is now decoupled into RBPEX local recovery and RBPEX Write-Behind recovery. They run independently and concurrently.

The RBPEX local recovery loads the foreign redo information from remote RAI and starts foreign log apply from that point on. It uses Localstate to determine where to resume applying the logs. Local recovery ensures that the Page Server can replay logs and bring its local cache consistent with the database state.

The RBPEX Write-Behind recovery, starts a log scan at the remote RAI and goes forward until the local RAI and sets all the bits in the dirty page bitmap corresponding to log records in that range, but does not apply log to those pages because RBPEX already has been refreshed for that log range. The regular log scan uses the local RAI instead of the remote one to determine where to start applying log to the current Page Server. Recovery is executed in the following steps:

Once the Write-Behind thread is started, remote GFH is queried to fetch foreign redo info. If it is ahead of RemoteState, then update RemoteState to bring it in sync with GFH. This is done because RemoteState is used as the starting point of recovery and hence needs to be in sync with the remote GFH value.

The Write-Behind recovery routine is then executed which contains the core recovery logic to update the Write-Behind dirty page bitmap for pages in RBPEX that are ahead of remote file and have not been flushed. This involves the following steps:

- Initialize iterator that fetches log from the log service.
- Log scan is then initialized to execute from start LSN = RemoteState and end LSN = LocalState.
- For each log record fetched between start and end LSN's, if it contains a page update, the page id is retrieved that needs to be updated in Write-Behind page modified bitmap.
- For each such page, we modify the corresponding bit in the PagesModified bitmap.

Once the above steps for write-behind recovery are completed successfully, the database recovery process is marked as complete. At this point, the write-behind checkpoint thread resumes its normal execution cycle, ensuring that subsequent checkpoints and flush operations proceed without interruption. This finalization step guarantees transactional consistency and prepares the system for steady-state operations following recovery.

In summary, the Write-Behind RBPEX approach mitigates resiliency challenges arising from Azure Storage outages and performance constraints by decoupling write operations between RBPEX and Azure Storage. Instead of synchronous dual writes, data modifications are first captured in RBPEX memory buffers and asynchronously flushed to Azure Storage by a background thread. This design ensures uninterrupted Page Server operation even when Azure Storage experiences latency or downtime. By maintaining local and remote metadata consistency through RBPEX-level Apply Information (RAI) tracking and enforcing the invariant LocalState ≥ RemoteState, data integrity is preserved. The asynchronous checkpoint mechanism further enhances throughput, reduces write latency, and ensures robust recovery. Overall, Write-Behind RBPEX ensures high availability and storage independence without compromising durability.

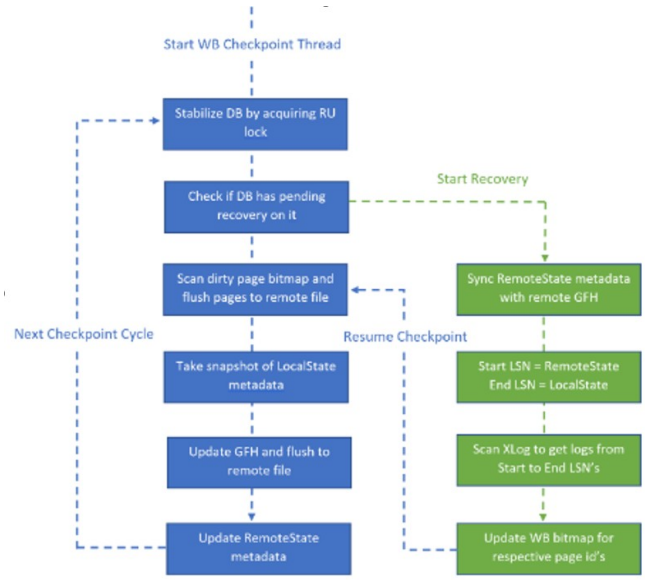


Figure 9: Write-Behind recovery path

5 Performance Experimentation

This section presents experimental results that assess the effectiveness of the storage improvements to Hyperscale. For comparison, the baseline Hyperscale architecture described in Section 2 is used as the reference point.

5.1 Experiment Setup

The evaluation focused on two key metrics: IO operations per section on Azure Storage and Azure Storage throughput during workload execution.

Four experimental configurations were tested:

- Baseline with no optimizations enabled
- Enabling only the logical-physical file separation optimization
- Enabling only the Write-Behind optimization, and

- Enabling all available optimizations

Since these optimizations primarily stress the storage layer—specifically the Page Servers—a custom in-house workload was employed that combines heavy updates with intensive reads to induce frequent page modifications. The aim of the workload was to cause the Page Servers to make frequent updates to its pages.

Performance testing was done using two workloads:

- TPC-C Benchmark via HammerDB [5], an industry-standard OLTP performance benchmark executed on 8000-warehouse database (approximately 1TB in size) using a 32 v-core Compute configuration. The workload was run for 60 minutes with 256 virtual users.
- CDB (Microsoft's Cloud Database Benchmark), a synthetic workload operating on a six-table database with configurable transaction distributions, widely used to validate DBaaS offerings in Azure. This test ran on a 1 TB CDB database with a 32 v-core Compute replica.

Table 3 shows the improvements in Azure Storage IOPS and throughput as new optimizations are added and clubbed together for both workloads mentioned above:

Table 3: Improvements to performance

Storage Tier Modifications	Workload	Azure Storage IOPS	Azure Storage Throughput
Baseline	TPC-C	6717	60.47 MB/sec
	CDB	7724	85.67 MB/sec
With separating physical and logical files into 8 slices	TPC-C	7632 [954 per slice]	64.45 MB/sec
	CDB	8504 [1063 per slice]	92.66 MB/sec
With Write-Behind optimization	TPC-C	3874	58 MB/sec
	CDB	3958	90.59 MB/sec
With separation in 8 slices + Write-Behind	TPC-C	2752 [344 per slice]	67.46 MB/sec
	CDB	2880 [360 per slice]	95.04 MB/sec

The lower the IOPS, the better it is overall for system performance. Table 3 shows that with each new improvement, the IOPS went down especially with both improvements together in the final row. With the logical-physical file separation the IOPS are distributed over 8 slices, so each slice only has about 954 requests per second (85.8% reduction with respect to baseline) in case of TPC-C workload and 1063 requests per second (82.3% reduction with respect to baseline) for CDB workload. Since Write-Behind checkpoint combines the IOs, it ends up causing the biggest difference in reducing the number of requests to Azure Storage.

Table 4 shows the number of Page Servers for which the Write-Behind checkpoints were lagging during the Azure Storage outage.

Table 4: Resiliency during Azure Storage outage

Page Servers With Write-Behind Checkpoint Lagging	Write-Behind Checkpoint Lag
155	30–60 GB
79	60–90 GB
29	90–120 GB
39	120–150 GB
61	150–180 GB
2	180–210 GB

The above table shows that for around 365 Page Servers the Write-Behind checkpoint was lagging while the Page Servers were still caught up and available during a Azure Storage unplanned outage in Production. Due to the decoupling of IO operations as part of the Write-Behind improvement, Page Servers were able to apply logs to its own RBPEX and remain available.

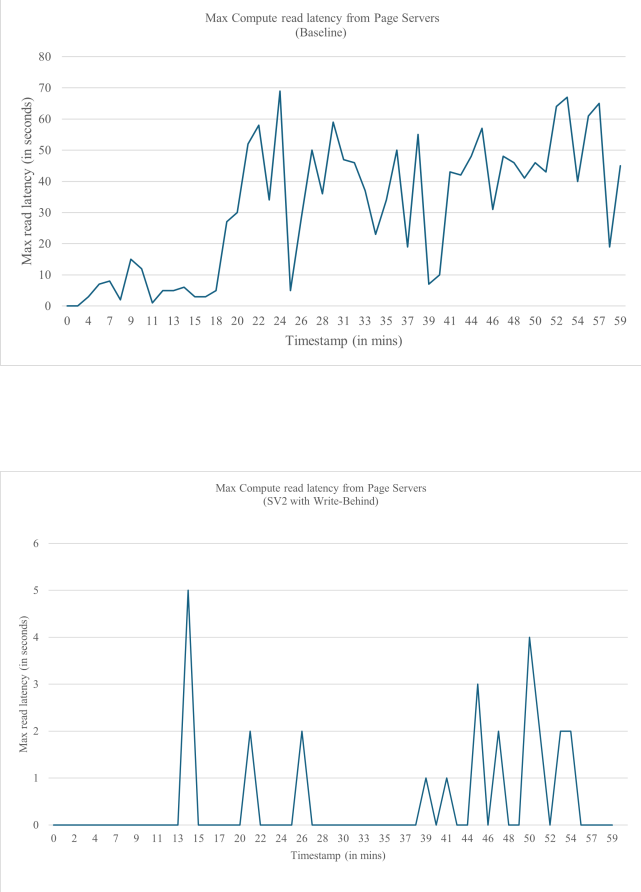


Figure 10: Max Compute read latency from Page Servers

As depicted in Figure 10, the new improvements clearly help in reducing the read latencies from Compute to the Page Servers. The decoupled IO (RBPEX IO separated from Azure Storage IO) helps in the write IOs to be harden faster on RBPEX whereas the logical-physical file separation distributes the IO allowing reads to not conflict with ongoing writes.

These experiments also prove that lower-level storage optimizations can reduce disruption of customer workloads due to external factors as well as create a more efficient and performant system.

6 State of the Art in Cloud-Native Database Architectures

Cloud-native relational databases increasingly decouple compute from storage to improve elasticity and availability. Amazon Aurora adopts a 6-way replicated, quorum-based storage service and offloads redo processing to the storage tier to reduce network IO and eliminate synchronous page writes across the network. The SIGMOD'17 paper motivates this design by analysing how traditional quorum and multi-phase commit protocols (e.g., 2PC) can introduce synchronous stalls and additive latency at cloud scale; Aurora's approach seeks to avoid these bottlenecks via asynchronous redo shipping and resilient quorum design (e.g., 4/6 write quorum across three AZs) [11]. Aurora's follow-on work further explains avoiding distributed consensus for IOs and commits under most circumstances by establishing invariants and leveraging local transient state, again targeting performance variability and coordination overheads common in quorum-based systems [12].

Alibaba Cloud PolarDB explores complementary directions. Recent PVLDB and SIGMOD/Industry publications describe RDMA-accelerated log shipment, strongly consistent reads in read-write splitting clusters (PolarDB-SCC), and multi-primary designs (PolarDB-MP) built on disaggregated shared memory and storage, addressing write scale-out while controlling conflict resolution and buffer fusion overheads. These works report low-latency strong consistency for RO nodes and competitive throughput under high-conflict workloads, positioning PolarDB as an alternative to quorum-centric coordination paths [14], [13], [1].

Azure SQL Hyperscale follows the same broad idea of separating compute from storage, but it makes different design choices around write coordination and storage parallelism. In Hyperscale, storage tier is defined as Page Servers which owns the subset of database pages and acts as the intermediary between compute replicas and durable remote storage. This paper describes an evolved storage tier that further breaks large logical files into dynamically managed slices, cells, and stripes, increasing IO parallelism and reducing per-object contention. In addition, asynchronous write-behind through RBPEX separates local persistence from remote durability, shortening the write critical path and making the system more resilient to transient storage slowdowns. Overall, Hyperscale emphasizes scalability through parallelism and decoupling, rather than relying on centralized quorum protocols or shared-memory coordination.

Many industrial cloud-native relational databases now follow a similar pattern of disaggregating compute from storage. Database systems such as Google AlloyDB and the open-source Neon database are built around the same core idea of separating stateless compute from durable remote storage, using local caching to achieve elasticity and fast recovery. Public documentation and engineering blogs highlight design choices such as log-centric remote persistence, rapid compute provisioning, and largely asynchronous interaction with durable storage, which help reduce tight coordination on the critical path. While the absence of peer-reviewed publications limits deeper architectural comparison, these systems nevertheless reflect a broader industry trend toward disaggregated architectures as the foundation for scalable relational databases.

7 Conclusion

This paper introduces two complementary storage enhancements for Hyperscale database that improve its performance and resiliency. Our evaluation results show that Hyperscale with both logical-physical file separation and RBPEX Write-Behind can handle demanding workloads efficiently in all environments.

The redesigned Hyperscale Storage Architecture fundamentally modernizes the data organization by decoupling logical and physical layers and employing granular distribution across slices and cells. This enables faster replica re-creation, higher resiliency, and linear IO scalability for different workloads. Its distributed topology eliminates single points of failure while ensuring stable, predictable throughput under varied load conditions.

In parallel, the Write-Behind RBPEX enhancement optimizes the write path through asynchronous data persistence. By deferring and aggregating writes to Azure Storage blobs, it reduces write latency, improves recovery reliability, and enhances resource utilization. The checkpoint and recovery components further ensure robust continuity even under transient hardware or process failures.

The enhancements discussed in this paper collectively establish a robust foundation for resource elasticity within the Hyperscale database architecture. By enabling dynamic allocation of computational resources across the storage layer and facilitating resource sharing among multiple databases, these improvements advance scalability and operational efficiency. Future work will focus on differentiating hot and cold storage tiers, a strategy aimed at optimizing performance while reducing COGS. These initiatives underscore a broader commitment to adaptive resource management and cost-effective cloud database solutions.

Acknowledgments

This work is supported by Naveen Prakash, Denzil Ribeiro, Ameya Sakhalkar, Alexey Eksarevskiy, Hugh Qu, Sheetal Shrotri, Daniel Lemmond, Subhodeep Maji and Kaleab Nigusie.

References

[1] Alibaba Cloud. 2025. PolarDB: RDMA-based Log Shipping and Shared Storage Architecture. https://www.alibabacloud.com/help/en/polardb/polardb-for-mysql/user-guide/rdma-log-transfer Documentation page; conference location: El Paso, Texas, USA.

[2] Panagiotis Antonopoulos, Alex Budovski, Cristian Diaconu, Alejandro Hernandez Saenz, Jack Hu, Hanuma Kodavalla, Donald Kossmann, Sandeep Lingam, Umar Farooq Minhas, Naveen Prakash, Vijendra Purohit, Hugh Qu, Chaitanya Sreenivas Ravella, Krystyna Reisteter, Sheetal Shrotri, Dixin Tang, and Vikram Wakade. 2019. Socrates: The New SQL Server in the Cloud. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1743–1756. doi:10.1145/3299869.3314047

[3] Brad Calder, Ju Wang, Aaron Ogus, Niranjan Nilakantan, Arild Skjolsvold, Sam McKelvie, Yikang Xu, Shashwat Srivastav, Jiesheng Wu, Huseyin Simitci, Jaidev Haridas, Chakravarthy Uddaraju, Hemal Khatri, Andrew Edwards, Vaman Bedekar, Shane Mainali, Rafay Abbasi, Arpit Agarwal, Mian Fahim ul Haq, Muhammad Ikram ul Haq, Deepali Bhardwaj, Sowmya Dayanand, Anitha Adusumilli, Marvin McNett, Sriram Sankaran, Kavitha Manivannan, and Leonidas Rigas. 2011. Windows Azure Storage: a highly available cloud storage service with strong consistency. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles* (Cascais, Portugal) *(SOSP '11)*. Association for Computing Machinery, New York, NY, USA, 143–157. doi:10.1145/2043556.2043571

[4] Cristian Diaconu, Craig Freedman, Erik Ismert, Per-Ake Larson, Pravin Mittal, Ryan Stonecipher, Nitin Verma, and Mike Zwilling. 2013. Hekaton: SQL server's memory-optimized OLTP engine. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data* (New York, New York, USA) *(SIGMOD '13)*. Association for Computing Machinery, New York, NY, USA, 1243–1254. doi:10.1145/2463676.2463710

[5] HammerDB Project. 2025. HammerDB: Database Load Testing and Benchmarking Documentation. https://www.hammerdb.com/document.html Online documentation; accessed 2025.

[6] Microsoft. 2025. Azure Premium Storage: Design Guidelines for High-Performance Applications. https://learn.microsoft.com/en-us/azure/virtual-machines/premium-storage-performance Microsoft Learn documentation; accessed 2025.

[7] Microsoft. 2025. *Hyperscale Service Tier.* https://docs.microsoft.com/en-us/azure/azure-sql/database/service-tier-hyperscale Azure SQL Database documentation.

[8] Barzan Mozafari, Carlo Curino, Alekh Jindal, and Samuel Madden. 2013. Performance and resource modeling in highly-concurrent OLTP workloads. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data* (New York, New York, USA) *(SIGMOD '13)*. Association for Computing Machinery, New York, NY, USA, 301–312. doi:10.1145/2463676.2467800

[9] OLTPBenchmark. 2025. OLTP Benchmark. http://oltpbenchmark.com. Accessed 2025.

[10] Rogerio Ramos, Prashanth Purnananda, Hanuma Kodavalla, Chaitanya Gottipati, Harshil Ambagade, Ankit Anvesh, and Srikanth Sampath. 2025. Hyperscale Resilient Buffer Pool Extension in Azure SQL Database. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, Hong Kong, Hong Kong, 4222–4233. doi:10.1109/ICDE65448.2025.00315

[11] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, and Xiaofeng Bao. 2017. Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases. In *Proceedings of the 2017 ACM International Conference on Management of Data* (Chicago, Illinois, USA) *(SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 1041–1052. doi:10.1145/3035918.3056101

[12] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, James Corey, Kamal Gupta, Murali Brahmadesam, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvilli, and Xiaofeng Bao. 2018. Amazon Aurora: On Avoiding Distributed Consensus for I/Os, Commits, and Membership Changes. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) *(SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 789–796. doi:10.1145/3183713.3196937

[13] Xinjun Yang, Yingqiang Zhang, Hao Chen, Feifei Li, Bo Wang, Jing Fang, Chuan Sun, and Yuhui Wang. 2024. PolarDB-MP: A Multi-Primary Cloud-Native Database via Disaggregated Shared Memory. In *Companion of the 2024 International Conference on Management of Data* (Santiago AA, Chile) *(SIGMOD '24)*. Association for Computing Machinery, New York, NY, USA, 295–308. doi:10.1145/3626246.3653377

[14] Xinjun Yang, Yingqiang Zhang, Hao Chen, Chuan Sun, Feifei Li, and Wenchao Zhou. 2023. PolarDB-SCC: A Cloud-Native Database Ensuring Low Latency for Strongly Consistent Reads. *Proc. VLDB Endow.* 16, 12 (Aug. 2023), 3754–3767. doi:10.14778/3611540.3611562